KANERVA: Token-Budgeted, Validity-Aware Memory for Coding Agents, Without an LLM in the Loop

Artem Katolikov*

October 2026

Abstract

Coding agents forget everything between sessions, and the memory systems built for them mostly select context by similarity top-k, ignore what items cost in tokens, and have no model of whether a stored fact is still true. We cast agent memory as online budgeted selection, show that the surveyed systems are special cases, and derive where each is worst-case suboptimal. We then present KANERVA, a local memory layer with no LLM in the loop. It separates an item's *validity* from its *usefulness*, selects under a hard token budget with a monotone submodular objective and a per-query optimality certificate, estimates validity from the store's own version history, and learns a linear policy from execution feedback. We prove a $\frac{1}{2}(1-\frac{1}{e})$ selection guarantee, a dynamic regret bound that separates the cost of expiring items as accumulated validity error, a linear lower bound for validity-blind learners, a rate–distortion bound for reversible consolidation, and a stationary store-size result. On retrieval-level metrics, with no LLM in the evaluation, KANERVA raises recall at 1024 tokens from 0.759 to 0.849 on LongMemEval-S and from 0.640 to 0.713 on LoCoMo against hybrid retrieval. On RepoMem, a new benchmark built from real repository histories, similarity retrieval injects an outdated version on 0.993 of queries about changed facts; KANERVA reduces this to 0.286 without structured keys, though a simple keep-the-newest-twin rule does as well, and neither helps on natural-language updates. A cross-encoder beats KANERVA at small budgets, hybrid retrieval wins at large budgets on LoCoMo, and forgetting, aggressive consolidation and noisy online learning all cost recall. We report these cases in full.

1 Introduction

A coding agent starts each session knowing nothing it learned in the last one. Native memory is a set of files loaded whole: Claude Code loads its `CLAUDE.md` files and the first 200 lines or 25 KB of a memory index, with no ranking by the task at hand [4]. Memory products add retrieval, but they select by similarity top-k or leave the choice to the agent, and they spend LLM calls on every write. The published evidence does not show that this is good enough. On LoCoMo, Mem0's own table has a full-context baseline ahead of every memory system tested [12]. Retrieval method moves accuracy far more than write strategy, and raw chunks with no LLM calls match fact extraction [69]. On coding tasks, unfiltered experience gives little or negative benefit [75], and eleven of twelve pairings of solvers with memory systems fail to beat memory-off [16].

These findings point at the read path, and at three defects in it. Selection ignores what an item costs in tokens. It ignores what the other selected items already say. And it ignores whether an item is still true, which is the common failure in a codebase, where a function signature read last month may have changed since.

This paper treats memory as an online budgeted selection problem and makes four contributions.

1. A framework in which the surveyed systems are special cases, with elementary worst-case arguments for what each one gives up (Section 3).

2. A method that separates *validity* from *usefulness*: budgeted monotone submodular selection with a per-query optimality certificate, a validity model estimated from the store's own version history, and a linear policy learned from execution feedback (Section 4). It runs locally and needs no LLM.

3. A regret decomposition for items that expire (Theorem 2). Stationary linear submodular bandit results exist [57, 68, 70]; they exclude this case. Here non-stationarity costs the accumulated validity error, while a validity-blind learner has linear regret (Proposition 8). The other theory results are a rate–distortion bound for reversible consolidation and a stability result for store size.

4. RepoMem, a coding-memory benchmark built from the real histories of eight Python repositories, in which facts change for real and every label comes from the code by static analysis. We evaluate on it, on LongMemEval-S and on LoCoMo

*Drafted with Claude Code. Code, data pointers and every reported number are at the project repository; see the Reproducibility section.

against a dozen reading baselines and Mem0's verbatim mode, with ablations, confidence intervals and the cases where the method loses.

Scope of the evidence. All results here are *retrieval-level*: evidence recall under a token budget, tokens injected, staleness errors, latency and store size. No LLM was available for these experiments, so we report no answer accuracy and no end-to-end task success. Recall under a budget is a necessary condition for those outcomes, not a measurement of them. A controlled study found that context strategy moved correctness on coding tasks by no more than 10 to 15 points [28], so large end-to-end gains should not be assumed.

2 Related work

Memory systems. Memory layers include OS-style paging by the agent [43], LLM-curated fact stores [12], temporal knowledge graphs with bi-temporal edge invalidation [49], linked notes [67], graph retrieval [15, 20, 21], and recursive summary trees [51]. Generative Agents scores memories by recency, importance and relevance [45]. Experience memories such as Reflexion, Voyager, ACE and ReasoningBank keep lessons or skills [42, 52, 63, 71]. All of these use a write path with LLM calls, a read path with a fixed k or an agent decision, or both. Of these systems, Zep/Graphiti is the one with a mechanism for validity. It is binary and depends on the extractor producing the invalidating edge.

Budgeted selection. Token-budgeted submodular selection for context is an active preprint line. PACMS applies facility location under a knapsack as a pluggable engine and lists learned relevance as future work [19]. AdaGReS scores relevance against redundancy [46], and PC-SubMax compresses prompts with a regularized objective [74]. The theory rests on Nemhauser et al. [41], on budgeted maximum coverage [29], on partial enumeration [17, 56], and on cost-benefit greedy with lazy evaluation [30, 34, 39]. MMR [8] and DPPs [31] target diversity without a budget in tokens.

Learning. Combinatorial and linear semi-bandits give regret with approximate oracles [9, 10, 33, 64], and linear submodular bandits extend this to diversified retrieval and to knapsacks [57, 68, 70]. Our analysis uses the confidence sets of OFUL [1]. In agent memory, DeMem proves regret bounds for a K-state memory partition [76] and MemRL shows Q-value convergence [72]; RSCB-MC and MERA use bandits to choose agent actions in coding [24, 25]. None of these treats token-budgeted selection of items whose validity changes. On reward signals, self-scored episodes inflate wrong answers [5], so we use execution signals.

Validity and forgetting. ACT-R base-level activation models the power law of forgetting [2, 3], and it has been applied to LLM memory [22]. Staleness is a measured failure: with a budget of two records, models acted on stale records in most episodes of one controlled study [40], and conflict benchmarks exist for chat [59]. Consolidation follows complementary learning systems [32, 38]. Rate–distortion views of compaction appear in [13, 53, 76]. Colaco and Lahjouji argue that repeated irreversible summarization degrades end-task error faster than retrieval-backed memory, which agrees with the context collapse reported for ACE [71], and motivates keeping sources.

Compression and parametric memory. Text compression such as LLMLingua [26, 27, 44] works with closed APIs and is complementary to selection. Gist tokens, AutoCompressors, ICAE, and KV-cache eviction [11, 18, 36, 47, 66, 73] and test-time weight updates [6, 7, 54] need access to model internals that closed APIs do not expose.

Benchmarks. LoCoMo [37], LongMemEval [65], MemoryAgentBench [23] and BEAM [60] cover conversational memory. SWE-ContextBench [75] and VibeMemBench [16] cover coding experience, MemoryCode covers multi-session coding instructions on synthetic data [48], and ContextBench covers retrieval within a task [35]. None of these, as far as we found, tests repository facts that change between sessions over real history.

3 A unified framework

Memory. At time t the store is a finite set M_t. An item is a tuple $m = (\text{text}, e, \tau^{\text{c}}, \tau^{\text{v}}, \pi, c, k, P)$: content, unit embedding, creation time, last time the item was confirmed, provenance, token cost $c(m) \in \mathbb{N}_{>0}$, an optional *slot key* k that identifies the fact the item states, and parent links P to the items it was consolidated from. Items sharing a key form a version chain.

Validity and usefulness. Fix a step context (query) q. Let $\rho_t(m) = \Pr[m \text{ still holds at } t]$ and $\mu_q(m) = \Pr[m \text{ helps with } q \mid m \text{ holds}]$. The *effective weight* of an item is $w_q(m) = \rho_t(m)\,\mu_q(m)$. Separating the two matters: ρ moves with time and changes in the world, μ with the query. Section 5 shows that a learner with no age or version information has linear regret.

Selection. Injecting $S \subseteq M_t$ has utility $U(S \mid q)$, and the context budget is a hard constraint:

$$\max_{S \subseteq M_t} \mathbb{E}\left[U(S \mid q_t)\right] \quad \text{s.t.} \quad c(S) := \sum_{m \in S} c(m) \le B. \quad (1)$$

We study the family

$$F(S) = (1-\gamma)\Big[\sum_{f\in\mathcal{F}_q}\omega_f\Big(1-\prod_{m\in S}\big(1-w(m)\,a_{mf}\big)\Big) + \eta\sum_{m\in S}w(m)\Big] + \gamma\sum_{c\in C}\frac{p_c}{\sum_{c'}p_{c'}}\max_{m\in S}s(c,m). \tag{2}$$

Here $\mathcal{F}_q$ is a set of facets of the query with weights $\omega_f \geq 0$, $\sum_f \omega_f = 1$, and $a_{mf} \in [0,1]$ is the chance that item m answers facet f given that it is valid and useful. The first term is the expected fraction of the need that gets covered when items succeed independently, the second the expected number of useful items injected, and the third a facility-location coverage of the candidate pool C with similarity $s \in [0,1]$ and fixed importance p_c, which removes near-duplicates.

Lemma 1. *For fixed $w \in [0,1]^M$, F is non-negative, monotone and submodular, with $F(\emptyset) = 0$. It is coordinate-wise non-decreasing in w and $|F(S;w) - F(S;w')| \leq L\sum_{m\in S}|w(m) - w'(m)|$ with $L = (1-\gamma)(1+\eta)$.*

The online problem. A memory policy $\Pi = (\mathsf{W}, \mathsf{R}, \mathsf{C}, \mathsf{F})$ decides what to write, what to read, what to consolidate and what to forget. For a sequence of steps it earns

$$J(\Pi) = \mathbb{E}\sum_{t=1}^{T}\Big[U(S_t \mid q_t) - \kappa_{\mathrm{r}}\,c(S_t)\Big] - \kappa_{\mathrm{w}}\,\mathrm{WriteCost}(\Pi) - \kappa_{\mathrm{s}}\sum_t c(M_t), \tag{3}$$

subject to $c(S_t) \leq B$ for all t. Reading costs tokens on every step, writing costs LLM calls or latency, and the store costs retrieval time and noise. Learning, in this paper, is updating Π and M_t; model weights never change.

3.1 Existing methods as special cases

Table 1 places the surveyed systems in (1)–(3). Nearly all read policies reduce to the modular case $\eta = 1, \gamma = 0$, $\mathcal{F}_q = \emptyset$ with a cardinality constraint and a method-specific score. The systems differ mostly in what they store, which is the W component.

Parametric methods (test-time training [54, 55], memory layers [7], Titans [6], gist tokens and KV-cache eviction [36, 47, 73]) keep memory in weights or attention state. They fall outside the framework by construction, and they require write access to weights or to the cache, which a closed API does not expose.

3.2 Where the special cases are suboptimal

The following statements are worst-case and elementary; the point is that each identifies a design defect that a better method must remove. Proofs are in Appendix A. Write OPT for the optimum of (1).

Proposition 1 (Cost blindness)**.** *Any rule that fills the budget in the order of a score independent of c (top-k fill, MMR, reciprocal rank fusion, reranking) has worst-case ratio at most $1/\lfloor B/c_{\min}\rfloor$ against* OPT*, even for additive U.*

Proposition 2 (Redundancy blindness)**.** *Any modular rule has worst-case ratio at most $1/k$ on coverage objectives with k facets when one facet has at least k near-duplicate items.*

Proposition 3 (Recency as proxy)**.** *A relevant item m followed by later items of total cost above B, each no larger than m, is never injected by recency-only selection, however relevant it is, whereas a rule that scores by relevance injects it whenever it outranks those items.*

Proposition 4 (Time-invariant scoring)**.** *Any selector whose score is invariant under permuting the timestamps of items with equal content features injects an outdated version ahead of the current one with probability at least $1/2$ on version pairs of equal text similarity to the query.*

Proposition 5 (Fixed policies)**.** *For any fixed linear scoring vector θ there is a $\theta^\star$ for which the fixed policy earns regret $\Omega(T)$, while the learner of Theorem 2 has regret $\tilde{O}(\sqrt{T})$ on the same instance.*

Table 1 shows which propositions apply: every k-limited row is cost-blind, every modular row is redundancy-blind, every row with no validity model or a recency proxy falls under Propositions 3–4, and every non-learning row under Proposition 5. The systems with invalidation (Zep/Graphiti) avoid Proposition 4 only for facts where the extractor produced the invalidating edge.

4 Method

Kanerva is the combination of the components below, implemented with SQLite, FTS5 and NumPy. No component calls an LLM. When a host agent is present it can supply structured writes through tools, which is how semantic extraction happens at no added API cost.

4.1 Write path

A write redacts secrets, deduplicates against live items (an identical write only refreshes the confirmation time

Table 1: Surveyed methods as instances of the framework. "Score" is the w used by the read policy; k denotes a cardinality limit.

Method	Write W	Score w	Constraint	Validity ρ	Learns
Top-k RAG, hybrid, rerank [14, 50]	chunks	BM25/cosine/RRF/CE	k	none	no
MMR [8]	chunks	greedy rule, λ trade-off	k	none	no
PACMS, AdaGReS [19, 46]	chunks	facility location; relevance−redundancy	$c(S) \le B$	none	no
MemGPT/Letta [43]	agent tool calls	LLM decides paging	block sizes	none	no
Mem0 [12]	LLM add/update/delete	similarity	k	LLM rewrite	no
Zep/Graphiti [49]	LLM graph extraction	hybrid + rerank	k	$\rho \in \{0, 1\}$ by edge invalidation	no
A-MEM [67]	LLM notes and links	similarity + links	k	LLM note evolution	no
HippoRAG 2 [21]	OpenIE triples	personalized PageRank	k	none	no
RAPTOR [51]	recursive summaries	similarity on all levels	k or tokens	none	no
Generative Agents [45]	observations, reflections	recency+importance+relevance	k	recency proxy	no
ACE, ReasoningBank [42, 71]	LLM-curated lessons	similarity or whole playbook	k	none	via text
Claude Code files [4]	model-decided notes	constant (query-independent)	200 lines/25 KB	none	no
Full history	everything	constant	window size	none	no
LLMLingua family [26, 44]	—	—	compresses $c(m)$	none	no

Algorithm 1 RETRIEVE(q, B, t)

$C \leftarrow$ top-N of FTS5 $\cup$ top-N of dense search
Remove superseded items, and items with a newer near-twin, unless q asks for history
for $m \in C$ **do**
  $x_m \leftarrow$ features of (m, q); $\mu_m \leftarrow \mathrm{clip}_{[0,1]}\big(x_m^\top \hat{\theta} + \beta \|x_m\|_{V^{-1}}\big)$
  $\rho_m \leftarrow$ Lomax survival (4) at $t - \tau^{\mathrm{v}}(m)$
  $\rho_m \leftarrow \rho_m \prod_{m' \text{ newer}} \big(1 - \pi\, g(s(m, m'))\big)$ ▷ no key
  $w_m \leftarrow \rho_m \mu_m$
end for
$\mathcal{S} \leftarrow \{\text{GREEDY}(F, B, \varrho) : \varrho \in \{1, \frac{1}{2}\}\} \cup \{\arg\max_{c(m) \le B} F(\{m\})\}$
return $\arg\max_{S \in \mathcal{S}} F(S)$, and on request certificate (5)

Algorithm 2 GREEDY(F, B, ϱ)

$S \leftarrow \emptyset$; $A \leftarrow \{m : c(m) \le B\}$
while $A \neq \emptyset$ **do**
  $m \leftarrow \arg\max_{m \in A} \left[F(S \cup \{m\}) - F(S)\right] / c(m)^{\varrho}$; remove m from A
  if the gain is 0 **then break**
  end if
  if $c(S) + c(m) \le B$ **then** $S \leftarrow S \cup \{m\}$
  end if
end while
return S

τ^{v}), and stores the item with its FTS5 entry and embedding. A write with a key closes the version interval of the live item with that key: the old item keeps its text and gets τ^{end} and a pointer to its successor. Nothing is rewritten, so history stays queryable.

4.2 Read path

Algorithm 1 shows retrieval. Candidates are the union of the top-100 lexical and dense hits. Features combine rank and score signals, query-term coverage, ACT-R base-level activation [3], and an item's feedback posterior, in twelve features, plus three optional signals from a local reranker. Facets are the query's content terms with *idf* weights plus one facet for the query as a whole.

Only the density variant $\varrho = 1$ and the singleton carry the guarantee; the other runs can only raise F, because the maximum is taken over all runs.

4.3 Validity

For a class k of facts (for example, function signatures), the unknown change rate has a Gamma(α, β) prior. After n_k observed changes over E_k item-days of exposure, the posterior predictive probability that an item confirmed a days ago still holds is

$$\hat{\rho}_k(a) = \left(1 + \frac{a}{\beta + E_k}\right)^{-(\alpha + n_k)}. \tag{4}$$

The class hazards are estimated from the store's own version history, so stable classes stop decaying without any tuning. (4) is a Lomax tail, which is a power law for ages well beyond $\beta + E_k$; at smaller ages it behaves like an exponential. So the mixture argument behind this estimator gives the same tail as the power-law decay seen in human memory and in the environment [2]; see Proposition 9.

Keyed facts have exact version chains: a superseded item is dropped from the candidates, and a query with historical intent ("previously", "used to") brings old versions back with $\rho = 1$. Text without keys has no such chain. For it we use a heuristic: a newer item with similarity above 0.8 discounts an older one (the product in Algorithm 1), and a newer near-twin above 0.97 removes it. Without embeddings, the similarity comes from feature hashing. This rule fires only on near-identical text, so it suits structured notes and rarely fires on natural-language updates; Section 6 measures both cases.

4.4 Learning

The policy keeps a ridge estimate with prior mean θ_0 fitted on development data: $\hat{\theta}_t = \theta_0 + V_t^{-1} \sum_{(x,y)} (y - x^\top \theta_0) x$ with $V_t = \lambda I + \sum x x^\top$. Feedback $y \in [0, 1]$ says whether an injected item was used, and a used item counts as reconfirmed. Items known to be invalid at feedback time are skipped, since they say nothing about μ. The update is $O(d^2)$ by Sherman–Morrison per item and works offline. In the plugin, an injected item counts as used when its identifiers or paths appear in later tool calls or edits, and a failed test halves the reward, following the evidence that self-reports inflate reward [5]. Item-level counters always update, but policy learning is opt-in, because Section 6 finds it fragile under noise.

4.5 Consolidation and forgetting

Consolidation is reversible. Star clusters around a leader, with every member above a similarity threshold of 0.9 by default, are summarized extractively: density greedy on a sentence-level facility location with rate R, which yields the rate–distortion guarantee of Theorem 3. The summary is a new semantic item with parent links, the sources move to an archive tier, and EXPAND restores them on demand. This follows the evidence that repeated irreversible rewriting erodes detail [13, 71]. Forgetting moves items to the archive when they have been idle for τ days or when the active store exceeds a token capacity, evicting the lowest $\rho \cdot \text{activation} \cdot \text{usefulness}/c$ first. Both rules are opt-in, because Section 6 finds that they cost recall. Deletion happens only on an explicit user command.

4.6 Multi-resolution

An item above a token threshold can be injected as a query-biased excerpt: the sentences that cover the most query terms, up to 40 tokens, marked with the item's id. Its cost in the knapsack is the excerpt's cost, so more items fit, and the agent pulls the full text with EXPAND only when it needs it. This mode is off by default.

4.7 Complexity

With N active items, $n \le 2N_c$ candidates, f facets, embedding dimension D, and $|S| = k$ selected: a write costs $O(\log N)$ plus one embedding; candidate generation costs $O(ND)$ by brute force (or sub-linear with an ANN index) plus the FTS5 query; greedy costs $O((k + r)\, n\, (f + n))$ per run with r rejected items, and lazy evaluation [34, 39] could reduce this further; building the similarity matrix costs $O(n^2 D)$. A bandit update costs $O(kd^2)$. Space is $O(N(D + \bar{c}))$. The memory layer itself uses no LLM tokens; the only token cost is the injected $c(S) \le B$.

5 Theory

All proofs are in Appendix A. Throughout, F is any set function with the properties of Lemma 1.

5.1 Selection

Theorem 1 (Selection guarantee)**.** *Let F be nonnegative monotone submodular with $F(\emptyset) = 0$ and let $c(m) \le B$ for all candidates. The set returned by Algorithm 1 satisfies $F(S) \ge \frac{1}{2}(1 - \frac{1}{e})\,\mathrm{OPT}$.*

This is the classical cost-benefit argument [30, 34], restated with a full proof because the constant in related analyses has needed correction [58]. Better constants exist for this algorithm family, 0.405 [58], and $1 - 1/e$ with partial enumeration at higher cost [17, 56]. The worst case is pessimistic for a practical reason: at each query the following bound can be computed.

Proposition 6 (Per-query certificate)**.** *For any feasible S, with $\Delta(m \mid S) = F(S \cup \{m\}) - F(S)$,*

$$\mathrm{OPT} \le F(S) + \max_{\substack{y \in [0,1]^M \\ \sum_m y_m c(m) \le B}} \sum_{m \notin S} y_m\, \Delta(m \mid S), \quad (5)$$

and the right-hand side is computed by sorting on Δ/c in $O(n \log n)$.

5.2 Learning with expiring items

The closest results [57, 68, 70] assume that an item's expected value is a stationary linear function of its features. Memory violates this: a fact's value collapses when the fact stops holding, while its text, and so its features, stay the same. We therefore model the two factors separately.

Setting. In round $t = 1, \ldots, T$ the learner sees candidates C_t with features $\|x_{t,m}\|_2 \le 1$ (rescaled by a constant if needed), costs, and a function $F_t(S; w)$ satisfying Lemma 1 with constant L. There is an unknown $\theta^\star$ with $\|\theta^\star - \theta_0\|_2 \le \Sigma$ and $\mu_{t,m} = x_{t,m}^\top \theta^\star \in [0, 1]$. Each item is valid with probability $\rho_{t,m} \in [0, 1]$ given the history before round t, which may vary arbitrarily with t, and the true weight is $w^\star_{t,m} = \rho_{t,m} \mu_{t,m}$. For each injected item the learner observes $Z_{t,m} \in \{0, 1\}$ with mean $\rho_{t,m}$, possibly correlated within a round, and, if $Z_{t,m} = 1$, a reward $y_{t,m} = \mu_{t,m} + \xi_{t,m}$, where $\xi_{t,m}$ is R-sub-Gaussian given the past, the current Z and the earlier rewards of the round. At most K items are injected per round. A validity estimate satisfies $|\hat{\rho}_{t,m} - \rho_{t,m}| \le \varepsilon_{t,m}$ for all t, m with probability at least $1 - \delta/2$.

Algorithm. Use $\bar{w}_{t,m} = \min(1, \hat{\rho}_{t,m} + \varepsilon_{t,m}) \cdot \mathrm{clip}_{[0,1]}(x_{t,m}^\top \hat{\theta}_t + \beta_t \|x_{t,m}\|_{V_t^{-1}})$, with $\hat{\theta}_t, V_t$ the prior-centred ridge estimates over the rounds where $Z = 1$, and pick S_t with an α-approximate oracle for $F_t(\cdot\,; \bar{w}_t)$ under the budget.

Theorem 2 (Validity-aware regret)**.** *Let $\lambda \geq K$ and $\beta_t = \beta_T := R\sqrt{d \log\big(4(1 + KT/\lambda)/\delta\big)} + \sqrt{\lambda}\,\Sigma$. With probability at least $1 - \delta$,*

$$\sum_{t=1}^{T} \Big[\alpha \max_{c(S)\leq B} F_t(S; w_t^\star) - F_t(S_t; w_t^\star)\Big] \leq L\Bigg[2\beta_T \sqrt{2dKT \log\Big(1 + \frac{KT}{d\lambda}\Big)} + K\sqrt{2T \log\frac{4}{\delta}} + 2\sum_{t=1}^{T}\sum_{m\in S_t} \varepsilon_{t,m}\Bigg].$$

The comparator is the per-round optimum under the time-varying $w_t^\star$, so the bound is dynamic, and the $\rho_{t,m}$ need no variation budget: the burden moves into estimating them, through the last term. The first term is the usual linear bandit price, the second comes from the randomness of validity, and warm starting helps through Σ. With $\alpha = \frac{1}{2}(1 - \frac{1}{e})$ from Theorem 1 and $L = (1-\gamma)(1+\eta)$, the theorem covers Algorithm 1 run with β_T and the margin ε.

What this does and does not say. The theorem describes how regret scales, and it is not a practical guarantee. Since $F \leq 1$, regret is trivially at most T, and with realistic constants ($K = 40$, $d = 12$) the bound falls below T only around $T \approx 10^7$ rounds. The shipped defaults use $\beta = 0$ and no margin, and they use a heuristic rule on top of (4), so the code is not the analyzed algorithm. The recall tables in Section 6 use the static prior and do not rest on learning at all.

Proposition 7 (Validity error of the hazard estimate)**.** *Let items of class k have lifetimes that are $\mathrm{Exp}(\lambda_k)$, with right-censoring that depends only on the past. Let N be the number of observed changes, E the total exposure up to a stopping time, and $\hat{\lambda}_k = N/E$. If $E \geq E_0$ almost surely and $\eta := 2\sqrt{\log(2/\delta)/(\lambda_k E_0)} \leq \frac{1}{2}$, then with probability at least $1 - \delta$, for all ages $a \geq 0$ simultaneously,*

$$|e^{-\hat{\lambda}_k a} - e^{-\lambda_k a}| \leq \frac{2}{e}\,\eta.$$

The bound does not depend on age: the error is governed by $\lambda_k E_0$, the expected number of observed changes. This estimator counts censored exposure, as the code does, whose posterior (4) has the same limit. If every class in use accumulates exposure as $\lambda_k E_0 \geq \nu t$, then $\varepsilon_{t,m} = \tilde{O}(1/\sqrt{\nu t})$ by a union bound over a grid of times, $\sum_t \sum_{m\in S_t} \varepsilon_{t,m} = \tilde{O}(K\sqrt{T/\nu})$, and the total is $\tilde{O}((d\sqrt{K} + K/\sqrt{\nu})\sqrt{T})$. For the conditional probabilities of the setting to match this survival curve, age must restart when an item is confirmed; the code does this for items that are used.

Proposition 8 (Validity-blind learners)**.** *Let $m \geq 2$. There is an instance in which m items per round have identical features, exactly one is valid, and B fits one item. Any learner that does not observe age or version information has $\mathbb{E}[\sum_t \alpha\,\mathrm{OPT}_t - F_t(S_t)] \geq (\alpha - \frac{1}{m})T$ for every $\alpha > 1/m$. So it has linear regret, for all horizons, whereas Theorem 2 applies with $\varepsilon \equiv 0$ if the version chain is known.*

5.3 Validity and activation

Proposition 9 (Power-law survival from a hazard mixture)**.** *If hazards in a class are $\mathrm{Gamma}(\alpha, \beta)$ distributed, the population survival is $S(a) = (1 + a/\beta)^{-\alpha}$. As $a \to \infty$, $\log S(a) = -\alpha \log a + O(1)$, the form of ACT-R single-trace base-level activation with decay $d = \alpha$.*

5.4 Consolidation

Let X be a cluster of sentences with weights p ($\sum_x p_x = 1$), costs c, cosine similarities $s \in [0,1]$ with $s(x,x) = 1$, and let $\Phi(Z) = \sum_x p_x \max_{z\in Z} s(x,z)$, $D(Z) = 1 - \Phi(Z)$ be the distortion of a summary $Z \subseteq X$.

Theorem 3 (Rate–distortion for extractive consolidation)**.** *Let Z_ℓ be the first ℓ picks of density greedy on Φ, with rate $R_\ell = c(Z_\ell)$. For every summary $Z^\star$ of rate $R^\star$,*

$$D(Z_\ell) \leq e^{-R_\ell/R^\star} + \big(1 - e^{-R_\ell/R^\star}\big) D(Z^\star).$$

Moreover, for every query vector e_q and every ℓ_{u}-Lipschitz item score $u_q(x) = \phi(\langle e_q, e_x\rangle)$,

$$\mathbb{E}_{x\sim p}\big[u_q(x) - u_q(z(x))\big] \leq \ell_{\mathrm{u}} \sqrt{2\,D(Z_\ell)}, \qquad z(x) = \arg\max_{z\in Z_\ell} s(x,z).$$

The token saving is $c(X) - R_\ell$, so distortion decays exponentially in rate against any reference summary. The statement covers the greedy prefix; the implementation continues past the first sentence that does not fit, as a heuristic. The bound averages over p and says nothing about a rare needle in the cluster. That is why consolidation here keeps the sources: if the summary is retrieved, EXPAND recovers any source at the cost of its tokens.

5.5 Store size

Theorem 4 (Store stability)**.** *Let items arrive as a Poisson process of rate a, and let each be accessed by an independent Poisson process of rate r, drawn i.i.d. from a distribution with $\mathbb{E}[(e^{r\tau} - 1)/r] < \infty$. An item is archived after an idle gap of τ. Then an*

item's active lifetime has mean $(e^{r\tau}-1)/r$ *given* r*, and the number of active items converges in distribution to* $\text{Poisson}\big(a\,\mathbb{E}[(e^{r\tau}-1)/r]\big)$.

Without forgetting, the store grows linearly in t. With an idle rule, the active store is bounded in distribution, and the retained set is biased exponentially toward frequently used items. The theorem says nothing about what forgetting costs in recall, and it assumes independent access processes, whereas real access patterns are bursty. Section 6 measures both.

6 Experiments

6.1 Setup

Mode. No LLM was available, so all metrics are retrieval-level. Hypotheses about end-to-end effects are out of scope.

Benchmarks. *LongMemEval-S* [65]: 500 questions, each with its own store of about 490 turns; 370 test questions remain after dropping those without flagged evidence and the dev split. *LoCoMo* [37]: 10 long conversations; conversations 0–2 are dev, leaving 1483 test questions. *RepoMem* (new): eight Python repositories at 14 snapshots over six years. Each interval is a session in which the simulated agent records commit notes and facts observed in changed files. Facts (constants, signatures, symbol locations, dependency constraints) come from `ast`, so a fact is stale exactly when the same key held another value at an earlier snapshot. Fact queries ask for the current value; task queries use a real later commit subject, with earlier commits on the same files as gold. Two repositories are dev, six test (4322 queries over three seeded streams each). We run RepoMem with slot keys, as a host agent would supply, and without. Four more repositories are a holdout (Section 6.5). A fourth track, *LongMemEval updates*, takes the 47 knowledge-update test questions with evidence in more than one session; the latest session's evidence is gold and earlier evidence is the outdated fact, in natural language.

Baselines. No memory (recall 0), full history (recall 1 at the cost of the whole store, a mean of 110,953 tokens per question on LongMemEval-S and 30,347 on LoCoMo), recency, BM25 (FTS5), dense (all-MiniLM-L6-v2 [61]), hybrid RRF [14], hybrid followed by a cross-encoder reranker, MMR [8], Generative Agents scoring [45] with constant importance, facility location under a knapsack as in PACMS [19], hybrid top-k for $k \in \{3, 5, 10, 20\}$, hybrid with superseded versions removed (temporal invalidation alone, in the spirit of [49]), hybrid listed newest first, and hybrid keeping only the newest of near-twins. All methods share candidate generators, token counts (`cl100k_base`) and a 14-token per-item overhead, the measured cost of an item's rendered prefix, and every ranking is filled greedily to the budget, which already makes the baselines budget-aware. Mem0 2.2.1 with `infer=False` and local Qdrant ran on a random 40-question sample. Systems that need an LLM on write were not run.

Protocol. The prior θ_0 and hyperparameters were set on dev splits only. Metrics are evidence recall, tokens injected, staleness (Section 6.3), latency and store size. Retrieval is deterministic; seeds govern RepoMem streams and online runs (five seeds). The independent unit is a question on LongMemEval, a conversation on LoCoMo (seven) and a repository on RepoMem (six), since seeds repeat many RepoMem queries and queries within a conversation are dependent. We give bootstrap intervals on the unit, and query-level intervals as well, which are optimistic for clustered data. p-values are two-sided sign-flip permutation tests on the mean at the unit level, with Holm correction across rivals and budgets for each reference method. With six units, the smallest possible p is $1/32$, so no RepoMem comparison can survive correction; there, read the unit interval and the count of units favoring each side.

6.2 Recall against tokens

Figure 1 shows the fronts and Table 2 the values. KANERVA here uses only BM25 and a small embedding model, and the recall tables use the static prior with no learning. On LongMemEval-S it leads hybrid and BM25 by a wide margin in the middle of the range: at 1024 tokens recall is 0.849 against 0.759 and 0.746. Most of that comes from cost density, and without it recall falls to 0.775. This partly reflects the dataset: evidence turns average 82 tokens and other turns 211, since evidence is mostly in short user turns, so density favors the right items. On LoCoMo the relation runs the other way, 46 against 30 tokens, and density hurts slightly. There KANERVA leads hybrid up to 1024 tokens (0.713 against 0.640), is level with BM25 at small budgets, and loses to hybrid at large ones (0.824 against 0.856 at 4096).

RepoMem fact queries are close to saturated: at 1024 tokens every method in Table 5 is near or above 0.95 on them, BM25 included, so the differences between methods come from a small share of queries. There, KANERVA is level with the rivals or slightly behind: 0.888 against 0.903 for hybrid without keys, and 0.897 against 0.908 for hybrid with invalidation. Removing older near-twins also costs it on the task track, the only hard one, where events that look alike are not versions of one fact. On the holdout repositories without keys it has 0.809 against 0.835.

The strongest rival is the cross-encoder reranker, which spends a neural model on every query (median 220 ms for 50 pairs on this machine's GPU). It beats KANERVA at small budgets: 0.512 against 0.389 on LoCoMo at 128 tokens, and 0.697 against 0.629 on LongMemEval-S at 256. By 1024 tokens the two are level within their intervals.

Extension with reranker features. This result suggested feeding the cross-encoder's signals into the policy as three more features, with a prior refitted on dev. The variant was designed after seeing the test results above, so it is a post-hoc extension and needs confirming on fresh data. "KANERVA + reranker" narrows the gap (Table 4): against the plain reranker its unit-level intervals favor it at 3 of six budgets on LongMemEval-S and 2 on LoCoMo, and favor the reranker at 0 and 2. It needs a model that the default plugin does not load.

Lighter configurations. Embedding-free KANERVA, the plugin default, reaches 0.818 on LongMemEval-S and 0.686 on LoCoMo at 1024 tokens. With the static model2vec embedder [62] in place of MiniLM these are 0.858 and 0.705.

Mem0. On the 40-question sample, Mem0's verbatim mode reached 0.829 at 1024 tokens against 0.877 for KANERVA and 0.846 for hybrid on the same questions, with 9 ms per write. Mem0's LLM extraction, its main feature, was not exercised.

Tables 3 and 4 give the paired differences against the strongest rival at three budgets. The unit intervals on RepoMem rest on six repositories, so differences of a point or two there should be read as ties. Table 5 breaks recall down by question type.

6.3 Staleness

Table 6 reports, on queries whose fact changed, how often an outdated version is *injected*, and the *error*: how often it is injected and the current version is absent or ranked below it. Injection is the stronger test. Listing hybrid's selection newest first drops its error from 0.696 to 0.019 with no change to what is injected, 0.993.

The problem is real and large: similarity retrieval injects the outdated version nearly always on changed facts, and a larger budget does not help. The fix turns out to be cheap. With keys, exact version chains remove stale items entirely, and plain hybrid with invalidation does this as well as KANERVA, with slightly higher recall. Without keys, KANERVA injects a stale version on 0.286 of these queries, and on 0.539 on the holdout repositories, but the simple rule of keeping the newest near-twin reaches 0.315 and 0.632, with recall 0.897 on the test repositories. So on RepoMem the gain comes from that rule, which KANERVA also uses, and not from its probabilistic model. Removing the rule from KANERVA brings injection back to 0.997. We have no ablation that isolates the hazard model's value here.

The rule depends on near-identical text. RepoMem's versions differ only in a value, which favors it. On the natural-language updates of LongMemEval, the evidence for a fact and for its update are not near-twins, the rule does not fire, and KANERVA injects the outdated turn on 0.979 of questions against 0.894 for hybrid. Its error there is 0.489, against 0.468. Listing items newest first does help, taking hybrid's error to 0.064, so the plugin lists injected notes that way. Detecting that a free-text statement replaces an older one remains unsolved here, and it likely needs semantics that an LLM-free method lacks.

6.4 Ablations

Table 7 removes one component at a time. After correction, only cost density on LongMemEval-S at 1024 tokens is significant, and the clustered datasets have too few units to reach significance. Density is decisive there and slightly harmful elsewhere: on LoCoMo removing it adds up to 0.023 recall at small budgets. Validity has almost no effect on recall on chat benchmarks. On RepoMem it helps at small budgets, by freeing budget that stale versions would use, and costs about a point without keys at 1024 tokens; its main effect is on staleness, which recall does not show. Facets add less than a point. The facility-location term and activation have effects indistinguishable from zero. We keep them because they are cheap, and because activation only becomes informative with real access histories, which these benchmarks lack, but this benchmark set does not establish their value.

6.5 A disclosed post-hoc change

The first test run showed KANERVA below hybrid on keyed RepoMem, and the ablation pointed at soft supersession: signatures of different functions are near-duplicates, so they discounted each other. Since keyed items have exact version chains, we restricted soft supersession to items without a key. This fix was found on test data, so numbers on those six repositories are no longer clean. Table 2 reports both the first-run rule and the fix. To check it fairly we added four repositories after that run (starlette, typer, black, aiohttp), never used for tuning. There, at 1024 tokens, the fixed method has recall 0.833 against 0.819 for the first-run rule, 0.835 for hybrid and 0.842 for hybrid with invalidation. LongMemEval and LoCoMo have no keys, so the change does not touch them.

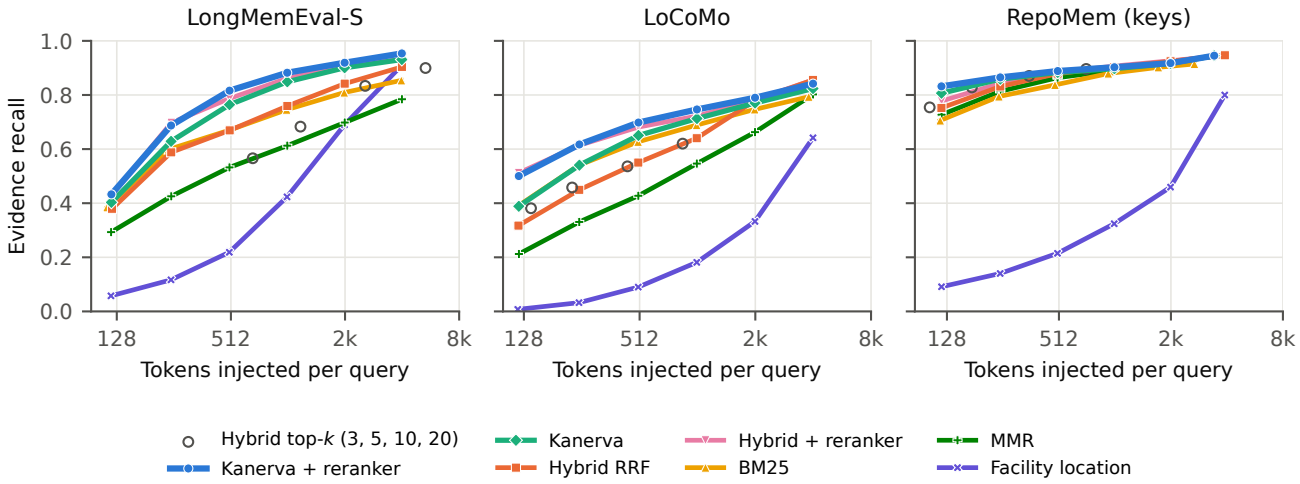


Figure 1: Evidence recall against mean tokens injected. Each curve visits budgets 128 to 4096; open circles are fixed top-k. Up and to the left is better.

6.6 Online learning

Feedback here is simulated from evidence labels, as described in Section 7. Each seed shuffles the order of work and shares a single policy; Table 8 reports recall at $B = 512$ over the second half of each run, and Figure 2 the learning curves. Learning from a fitted prior adds little: on LongMemEval-S recall moves from 0.769 to 0.768, on LoCoMo from 0.648 to 0.662, and on RepoMem from 0.900 to 0.901. Learning matters when the prior is wrong: from a flat prior, static recall is 0.706 on LongMemEval-S and 0.583 on LoCoMo, and the online policy recovers to 0.768 and 0.652, close to the fitted prior. The UCB bonus changes little. Noise is the danger. With 20% of rewards flipped, online learning does *worse* than not learning at all: 0.747 on LongMemEval-S, 0.585 on LoCoMo and 0.888 on RepoMem, because the flips raise the weight of the many irrelevant items. The standard correction for a known flip rate recovers part of the loss on LongMemEval-S and most of it elsewhere (0.757, 0.642 and 0.899), but a real deployment does not know its flip rate. So we ship policy learning as opt-in. We did not test whether a larger ridge would make it safe.

6.7 Checks of the theory on synthetic instances

With known ground truth (eight seeds, horizon 4000, 40 items per round, three classes with different hazards), cumulative regret against the per-round greedy optimum is 22 for a learner that knows validity, 26 when it estimates hazards from censored lifetimes, and 2015 when it is blind. The log–log slopes in T are 0.26, 0.25 and 0.97. The blind learner's linear growth is what Proposition 8 predicts, and it shows the value of the separation. The observed regret of the other two is far below the upper bound of Theorem 2, whose value here exceeds the trivial bound, so these runs do not test the bound's tightness. Over 1600 random selection problems, $F(S)$ was on average 0.707 of the certificate upper bound, with a minimum of 0.600, above the worst-case constant of 0.316.

6.8 Consolidation, forgetting and multi-resolution

Consolidation. Table 9 applies consolidation at several thresholds. A gold item counts as found if it or its summary is selected, which measures discoverability, since the sources stay expandable. Summaries are embedded like other items, and the LongMemEval-S sample is 60 random test questions. Consolidation is not free. At $\tau = 0.9$ it merges only near-duplicates: it saves 2.1% of tokens on the LongMemEval-S sample and 0.5% on LoCoMo, with recall unchanged. At $\tau = 0.8$ it saves 19.9% and 5.6%, and recall moves from 0.874 to 0.826 and from 0.713 to 0.711. At $\tau = 0.7$ the loss is larger. Since the budget caps injected tokens anyway, a smaller store buys little here, so the default threshold is 0.9.

Forgetting. On RepoMem streams, Table 10 compares keeping everything with idle-time rules and token capacities. Forgetting has a real cost on this benchmark. Idle rules of 365 and 180 days cut the active store to 5909 and 2938 tokens from 21492, as Theorem 4 leads one to expect, but recall falls from 0.897 to 0.728 and 0.561, because long-lived facts are read rarely and are still correct. Sessions in this benchmark are 168 days apart, so these idle rules are short in session terms, and

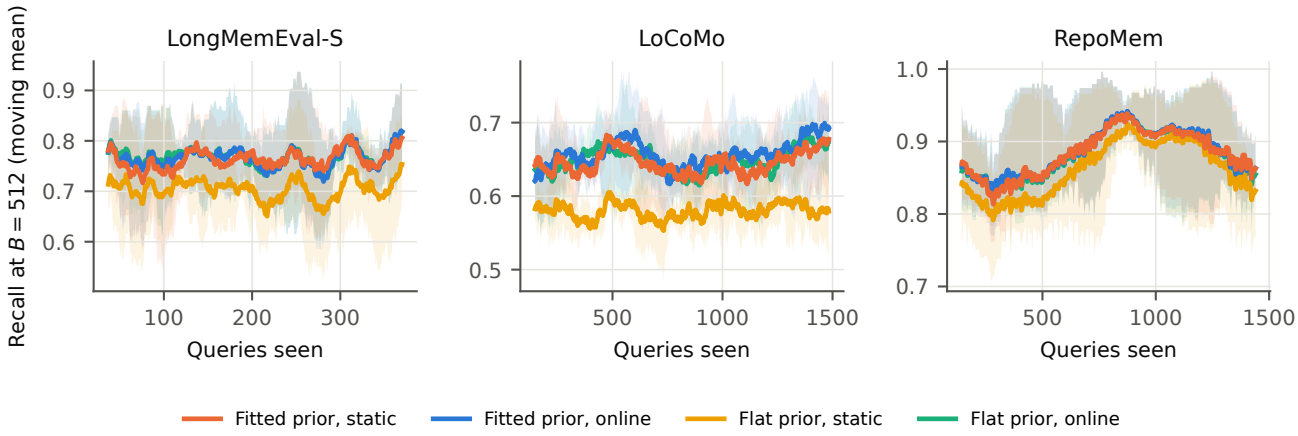


Figure 2: Online learning curves: moving-mean recall at $B = 512$. Bands span the seeds.

the numbers do not transfer directly to a daily workflow. Token capacities behave similarly. Given the latencies below, a store of this size does not need forgetting, so the rules are opt-in. Archived items remain recoverable by hand, but they leave retrieval.

Multi-resolution. Injecting query-biased excerpts of long items, with a pointer to the full text, raises the number of items that fit. On LongMemEval-S at 512 tokens, the share of evidence turns reached moves from 0.764 with full items to 0.833 with excerpts of items above 48 tokens, and at 1024 tokens from 0.849 to 0.906. This counts an excerpt as reaching the turn. Whether the excerpt contains the answer, or the agent would expand it, was not measured without an LLM.

6.9 Latency and scale

On an M1 Pro with an uncontended CPU, a fresh query at 10^4 items takes 53.6 ms at the median and 65.5 ms at the 95th percentile, plus 6 ms to embed the query, and lexical retrieval takes 27.7 and 36.6 ms. At 10^5 items the medians are 84.5 and 53.1 ms. A write costs 0.33 ms, excluding the embedding. The memory layer spends no LLM tokens, so its token cost is the injected context alone.

Table 2: Evidence recall at each token budget on test splits. Bold marks the best method per column within a dataset.

Method	128	256	512	1024	2048	4096
LongMemEval-S						
KANERVA	0.403	0.629	0.764	0.849	0.901	0.931
KANERVA + reranker	**0.433**	0.687	**0.816**	**0.882**	**0.919**	**0.954**
KANERVA, lexical	0.381	0.617	0.724	0.818	0.875	0.907
Hybrid + reranker	0.430	**0.697**	0.787	0.862	0.908	0.932
Hybrid RRF	0.379	0.588	0.669	0.759	0.842	0.904
BM25	0.385	0.599	0.669	0.746	0.809	0.854
Dense	0.327	0.497	0.592	0.687	0.771	0.882
MMR	0.293	0.425	0.532	0.613	0.698	0.784
Facility location	0.058	0.117	0.219	0.423	0.690	0.909
Generative Agents	0.202	0.288	0.347	0.400	0.452	0.520
Recency	0.000	0.003	0.003	0.018	0.030	0.054
LoCoMo						
KANERVA	0.389	0.541	0.650	0.713	0.770	0.824
KANERVA + reranker	0.500	**0.617**	**0.698**	**0.746**	**0.790**	0.842
KANERVA, lexical	0.388	0.533	0.628	0.686	0.748	0.793
Hybrid + reranker	**0.512**	0.614	0.681	0.722	0.782	**0.856**
Hybrid RRF	0.317	0.449	0.550	0.640	0.779	**0.856**
BM25	0.390	0.540	0.627	0.689	0.747	0.793
Dense	0.212	0.313	0.417	0.515	0.627	0.729
MMR	0.212	0.331	0.427	0.546	0.663	0.802
Facility location	0.008	0.032	0.090	0.181	0.333	0.642
Generative Agents	0.022	0.038	0.057	0.091	0.249	0.532
Recency	0.002	0.003	0.012	0.029	0.070	0.132
RepoMem, keys						
KANERVA	0.807	0.859	0.883	0.897	0.916	0.945
KANERVA + reranker	**0.832**	**0.865**	**0.889**	0.903	0.918	0.945
KANERVA, first run	0.784	0.850	0.879	0.896	0.917	0.946
KANERVA, lexical	0.769	0.821	0.860	0.885	0.904	0.913
Hybrid + reranker	0.778	0.843	0.883	0.907	0.926	0.947
Hybrid RRF	0.752	0.831	0.879	0.903	0.925	0.947
Hybrid, live only	0.814	0.856	0.886	**0.908**	**0.930**	**0.949**
Hybrid, newest of twins	0.804	0.851	0.882	0.902	0.925	0.943
BM25	0.706	0.794	0.837	0.879	0.902	0.915
Dense	0.784	0.847	0.876	0.900	0.919	0.939
MMR	0.727	0.813	0.864	0.891	0.924	0.947
Facility location	0.091	0.140	0.215	0.324	0.460	0.800
Generative Agents	0.199	0.276	0.456	0.641	0.837	0.928
Recency	0.009	0.025	0.046	0.093	0.199	0.333
RepoMem, no keys						
KANERVA	0.782	0.844	0.872	0.888	0.907	0.939
KANERVA, lexical	0.738	0.803	0.844	0.873	0.897	0.907
Hybrid RRF	0.756	0.831	**0.879**	**0.903**	**0.925**	**0.947**
Hybrid, live only	0.756	0.831	**0.879**	**0.903**	**0.925**	**0.947**
Hybrid, newest of twins	**0.798**	0.845	0.877	0.897	0.919	0.938
BM25	0.709	0.795	0.837	0.879	0.902	0.915
Dense	0.785	**0.847**	0.876	0.901	0.920	0.939
MMR	0.732	0.814	0.864	0.891	0.924	**0.947**
Facility location	0.091	0.142	0.218	0.329	0.467	0.808
Generative Agents	0.196	0.274	0.454	0.641	0.836	0.928

Table 3: KANERVA minus the best rival at each budget, in recall. Positive favors KANERVA. Unit CI resamples conversations or repositories; Units + counts those favoring KANERVA; p is Holm-corrected.

Dataset	Rival	B	Δ recall	Query CI	Unit CI	Units +	p
LongMemEval-S	Hybrid + reranker	256	-0.069	[-0.098, -0.040]	—	—	<0.001
LongMemEval-S	Hybrid + reranker	1024	-0.013	[-0.038, +0.011]	—	—	0.922
LongMemEval-S	Hybrid + reranker	4096	-0.001	[-0.014, +0.013]	—	—	0.952
LoCoMo	Hybrid + reranker	256	-0.073	[-0.091, -0.054]	[-0.082, -0.063]	0/7	0.938
LoCoMo	Hybrid + reranker	1024	-0.009	[-0.023, +0.005]	[-0.028, +0.005]	3/7	1.000
LoCoMo	Hybrid, newest of twins	4096	-0.032	[-0.045, -0.020]	[-0.041, -0.023]	0/7	0.938
RepoMem, keys	Hybrid, live only	256	+0.003	[-0.001, +0.007]	[-0.003, +0.010]	4/6	1.000
RepoMem, keys	Hybrid, live only	1024	-0.010	[-0.014, -0.007]	[-0.014, -0.007]	0/6	1.000
RepoMem, keys	Hybrid, live only	4096	-0.004	[-0.006, -0.002]	[-0.007, -0.001]	1/6	1.000
RepoMem, no keys	Dense	256	-0.003	[-0.009, +0.004]	[-0.019, +0.013]	2/6	1.000
RepoMem, no keys	Hybrid RRF	1024	-0.015	[-0.020, -0.011]	[-0.019, -0.011]	0/6	1.000
RepoMem, no keys	MMR	4096	-0.008	[-0.010, -0.005]	[-0.013, -0.004]	0/6	1.000
RepoMem holdout, keys	Hybrid, live only	256	-0.008	[-0.013, -0.003]	[-0.020, -0.000]	1/4	1.000
RepoMem holdout, keys	Hybrid, live only	1024	-0.009	[-0.015, -0.003]	[-0.022, +0.007]	1/4	1.000
RepoMem holdout, keys	Hybrid, live only	4096	-0.018	[-0.022, -0.014]	[-0.025, -0.013]	0/4	1.000
RepoMem holdout, no keys	Dense	256	-0.029	[-0.038, -0.019]	[-0.038, -0.019]	0/4	1.000
RepoMem holdout, no keys	Hybrid RRF	1024	-0.026	[-0.033, -0.019]	[-0.039, -0.015]	0/4	1.000
RepoMem holdout, no keys	Hybrid RRF	4096	-0.027	[-0.032, -0.022]	[-0.040, -0.017]	0/4	1.000

Table 4: As Table 3, for the post-hoc variant KANERVA + reranker.

Dataset	Rival	B	Δ recall	Query CI	Unit CI	Units +	p
LongMemEval-S	Hybrid + reranker	256	-0.010	[-0.023, +0.003]	—	—	0.405
LongMemEval-S	Hybrid + reranker	1024	+0.021	[+0.005, +0.037]	—	—	0.049
LongMemEval-S	Hybrid + reranker	4096	+0.022	[+0.009, +0.037]	—	—	0.011
LoCoMo	Hybrid + reranker	256	+0.003	[-0.007, +0.014]	[-0.005, +0.012]	3/7	1.000
LoCoMo	Hybrid + reranker	1024	+0.024	[+0.014, +0.036]	[+0.013, +0.037]	7/7	0.938
LoCoMo	Hybrid, newest of twins	4096	-0.014	[-0.023, -0.005]	[-0.022, -0.007]	0/7	0.938
RepoMem, keys	Hybrid, live only	256	+0.009	[+0.005, +0.014]	[+0.007, +0.012]	6/6	1.000
RepoMem, keys	Hybrid, live only	1024	-0.005	[-0.009, -0.002]	[-0.010, -0.001]	1/6	1.000
RepoMem, keys	Hybrid, live only	4096	-0.004	[-0.006, -0.002]	[-0.007, -0.001]	1/6	1.000
RepoMem holdout, keys	Hybrid, live only	256	+0.009	[+0.003, +0.016]	[-0.001, +0.018]	3/4	1.000
RepoMem holdout, keys	Hybrid, live only	1024	-0.005	[-0.010, +0.000]	[-0.020, +0.017]	1/4	1.000
RepoMem holdout, keys	Hybrid, live only	4096	-0.021	[-0.026, -0.017]	[-0.027, -0.016]	0/4	1.000

Table 5: Recall at $B = 1024$ by category.

Category	n	KANERVA	+ reranker	Hybrid	Rerank	BM25
knowledge update	48	0.969	1.000	0.875	0.990	0.896
multi session	101	0.741	0.830	0.648	0.827	0.613
single session assistant	45	0.911	0.911	0.889	0.889	0.889
single session preference	20	0.800	0.792	0.633	0.650	0.408
single session user	52	0.942	0.981	0.923	1.000	0.923
temporal reasoning	104	0.834	0.834	0.700	0.796	0.719
fact constant changed	2	1.000	1.000	1.000	1.000	1.000
fact constant stable	105	1.000	1.000	1.000	1.000	1.000
fact dependency changed	109	1.000	1.000	1.000	1.000	1.000
fact dependency stable	80	1.000	1.000	1.000	1.000	1.000
fact location changed	345	0.997	0.997	0.997	0.997	0.986
fact location stable	382	1.000	1.000	1.000	1.000	0.984
fact signature changed	1822	0.992	0.996	0.990	0.994	0.956
fact signature stable	719	0.999	0.999	0.999	0.999	0.997
task	758	0.437	0.456	0.474	0.487	0.435

Table 6: Staleness at $B = 1024$ on queries whose fact changed: share with an outdated version injected, and share with an error.

Method	RepoMem, keys		RepoMem, no keys		Holdout, no keys		LME updates	
	inject	error	inject	error	inject	error	inject	error
KANERVA	0.000	0.000	0.286	0.018	0.539	0.049	0.979	0.489
KANERVA, lexical	0.000	0.000	0.279	0.025	0.523	0.082	0.936	0.553
KANERVA, no soft supersession	0.000	0.000	0.997	0.694	0.987	0.641	0.979	0.553
KANERVA, no validity	0.997	0.702	0.997	0.694	0.987	0.641	0.979	0.553
Hybrid, live only	0.000	0.000	0.993	0.696	0.987	0.642	0.894	0.468
Hybrid, newest of twins	0.307	0.000	0.315	0.018	0.632	0.033	0.894	0.064
Hybrid, newest first	0.993	0.002	0.993	0.019	0.987	0.039	0.894	0.064
Hybrid RRF	0.993	0.705	0.993	0.696	0.987	0.642	0.894	0.468
Dense	0.999	0.455	0.999	0.454	0.985	0.449	0.851	0.489
BM25	0.968	0.765	0.968	0.758	0.949	0.699	0.915	0.596
Hybrid + reranker	0.998	0.485	—	—	—	—	—	—

Table 7: Ablations: recall of KANERVA minus the variant with a component removed, at $B = 256$ and 1024. A positive value means the component helps. *: $p_{\text{Holm}} < 0.05$.

Removed	LongMemEval-S		LoCoMo		RepoMem, keys		RepoMem, no keys		RepoMem holdout, keys		RepoMem holdout, no keys	
	256	1024	256	1024	256	1024	256	1024	256	1024	256	1024
no cost density	+0.011	+0.074*	-0.013	-0.010	-0.007	-0.003	-0.006	-0.003	-0.017	-0.004	-0.012	-0.007
no facets	+0.005	-0.004	+0.004	+0.003	+0.002	+0.000	+0.005	+0.002	+0.006	+0.002	+0.006	+0.003
no redundancy term	-0.005	+0.004	-0.001	-0.001	-0.001	-0.000	-0.000	+0.000	-0.001	+0.000	-0.001	-0.001
no validity	-0.006	-0.001	-0.000	+0.001	+0.023	-0.000	+0.008	-0.010	+0.014	+0.001	-0.017	-0.024
no soft supersession	-0.006	-0.001	-0.000	+0.001	-0.002	-0.006	+0.008	-0.009	-0.004	-0.004	-0.017	-0.023
no activation	-0.001	+0.007	-0.001	-0.002	+0.000	-0.001	-0.000	-0.002	-0.002	-0.001	-0.001	-0.003

Table 8: Prequential recall at $B = 512$ over the second half of each run, mean ± s.d. over five seeds. "Noisy" flips 20% of rewards. "Flat" starts from a uniform prior.

Dataset	Static	Online	UCB	Noisy	Debiased	Flat static	Flat online
LongMemEval-S	0.769 ± 0.021	0.768 ± 0.023	0.767 ± 0.024	0.747 ± 0.013	0.757 ± 0.021	0.706 ± 0.017	0.768 ± 0.024
LoCoMo	0.648 ± 0.018	0.662 ± 0.013	0.660 ± 0.015	0.585 ± 0.015	0.642 ± 0.010	0.583 ± 0.009	0.652 ± 0.013
RepoMem	0.900 ± 0.026	0.901 ± 0.027	0.902 ± 0.027	0.888 ± 0.028	0.899 ± 0.026	0.883 ± 0.029	0.897 ± 0.028

Table 9: Reversible consolidation on test splits (LongMemEval-S: 60 random questions). τ is the similarity threshold, R the extractive rate.

Dataset, τ, rate	Clusters	Sources	Tokens saved	Recall@1024
LoCoMo, baseline	0	0	0.0%	0.713
LoCoMo, $\tau = 0.7$, $R = 0.5$	577	1972	23.0%	0.672
LoCoMo, $\tau = 0.8$, $R = 0.5$	195	535	5.6%	0.711
LoCoMo, $\tau = 0.9$, $R = 0.5$	29	71	0.5%	0.716
LongMemEval-S, baseline	0	0	0.0%	0.874
LongMemEval-S, $\tau = 0.7$, $R = 0.5$	7985	20806	39.4%	0.728
LongMemEval-S, $\tau = 0.8$, $R = 0.5$	4625	10327	19.9%	0.826
LongMemEval-S, $\tau = 0.9$, $R = 0.5$	458	1015	2.1%	0.874

Table 10: Forgetting on RepoMem test streams: final active store and recall.

Policy	Active items	Active tokens	Recall@1024
keep-all	965	21492	0.897
idle-730d	536	11790	0.865
idle-365d	263	5909	0.728
idle-180d	131	2938	0.561
cap-8k	466	7987	0.765
cap-4k	279	3989	0.553

7 Limitations

No LLM in the evaluation. Every metric is retrieval-level. We do not know how much recall per token translates into correct answers or finished tasks. Evidence recall treats a turn as all-or-nothing, and a selected turn may still be hard for a model to use. Answer accuracy with an LLM is the obvious next measurement.

Baselines that need an LLM were not run. Mem0 with inference, Graphiti, A-MEM, HippoRAG, LangMem and Cognee all call an LLM on write. They could not run as designed, so the comparison does not cover them. Mem0 ran only in its verbatim mode (`infer=False`), on a subset. Our comparison says nothing about what LLM extraction would add to those systems, and published evidence suggests a small effect on the write side [69].

Validity evidence is narrow. The near-twin rule works on RepoMem because versions differ only in a value, and it does not fire on natural-language updates. It also treats look-alike events as versions, which costs recall on the task track; restricting it to facts is an obvious change we did not test. The hazard model has no ablation that isolates it, and detection of historical intent was not evaluated on any benchmark.

Feedback was simulated. The online experiments derive rewards from evidence labels. Real implicit signals, such as an item's identifiers reappearing in tool calls, are noisier and biased in ways that label flipping does not capture. The plugin's signal has not been validated against real sessions.

RepoMem is a proxy. Queries come from templates over extracted facts, so they contain identifiers and favor lexical search; real prompts are vaguer. The agent's notes are simulated from code and commit history. Every changed fact is assumed to be observed, so a store never misses a change. The task track uses file overlap as relevance, which is a weak label. Keyed mode gets perfect slot keys from the static analyzer, so it is an upper bound on what a real write path provides; plain mode is the conservative end. All repositories are Python libraries. Fact queries are nearly saturated, and the task track, where methods differ most, has weak labels.

Post-hoc steps. Several changes came after a look at test results: the keyed rule for soft supersession, removal of stale items, the reranker features, and the consolidation and forgetting defaults. An independent review also found a labelling error and a misleading metric, which led to a full rerun. Parameters were refitted on dev, but these designs were prompted by test results, so only the holdout repositories are clean for the supersession rules, and the reranker variant should be confirmed on fresh data.

The prior was fitted on the same benchmark families. θ_0 and the selection parameters come from dev splits of the three benchmarks, so test performance on a new domain may be lower. LongMemEval questions share distractor turns across splits, about 12% of turn text by one count, which is a small risk for a model with so few coefficients. KANERVA also benefits from this fit while most baselines have no tuned parameters.

Theory versus code. Theorem 2 assumes a linear μ, conditionally sub-Gaussian rewards and validity revealed with feedback. In practice validity is seen only when a successor appears, so unkeyed items that are stale enter learning with a reward of zero. As stated in Section 5, the bound is about scaling, and the defaults differ from the analyzed algorithm. Hazards assume exponential lifetimes within a class. The certificate bounds the surrogate F, not recall. Facets are bag-of-terms, so paraphrases only reach them through the query facet.

Token counts. We count with `cl100k_base` as a proxy for Claude's tokenizer, and the 14-token item overhead is an approximation of the rendered prefix.

Plugins. The plugins were tested with simulated hook payloads, the real entry script, and the plugin validator, but not in live Claude Code or OpenCode sessions.

8 Conclusion

Treating agent memory as budgeted selection over items with explicit validity gives a read path that needs no LLM and can state its optimality gap per query, and a theory in which expiring items have a clear price. On retrieval metrics, cost-aware selection raises recall per token where item costs vary and relevant items are cheap. A cross-encoder remains the stronger scorer at small budgets, and the objective can absorb it as a feature. Stale injection is a large problem on code facts, and simple rules fix most of it, so the probabilistic validity model has not yet shown value beyond them. Free-text updates remain unsolved. Plain hybrid stays better at large budgets on LoCoMo, and online learning helps mainly when the prior is wrong and is fragile under reward noise.

The next step is an end-to-end study: whether higher recall per token, and fewer stale injections, change what

a coding agent does. The retrieval results here are necessary for that outcome and do not establish it.

Reproducibility. `make data && make reproduce` rebuilds every result in `results/*.json` with commit hashes and seeds, and `make check` verifies each number in this paper against those files. The RepoMem manifest pins every repository snapshot.

References

[1] Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Improved Algorithms for Linear Stochastic Bandits. In *Advances in Neural Information Processing Systems 24*, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/e1d5be1c7f2f456670de3d53c7b54f4a-Abstract.html. Accessed 2026-10-30.

[2] John R. Anderson and Lael J. Schooler. Reflections of the Environment in Memory. *Psychological Science*, 2:396–408, 1991. doi: 10.1111/j.1467-9280.1991.tb00174.x. URL https://doi.org/10.1111/j.1467-9280.1991.tb00174.x. Accessed 2026-10-30.

[3] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An Integrated Theory of the Mind. *Psychological Review*, 111:1036–1060, 2004. doi: 10.1037/0033-295X.111.4.1036. URL https://doi.org/10.1037/0033-295X.111.4.1036. Accessed 2026-10-30.

[4] Anthropic. How Claude remembers your project, 2026. URL https://code.claude.com/docs/en/memory. Official docs. Accessed 2026-10-30.

[5] Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory Reward Inflation in Self-Improving LLM Agents, 2026. URL https://arxiv.org/abs/2608.00017. Accessed 2026-10-30.

[6] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to Memorize at Test Time, 2024. URL https://arxiv.org/abs/2501.00663. Accessed 2026-10-30.

[7] Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory Layers at Scale, 2024. URL https://arxiv.org/abs/2412.09764. Accessed 2026-10-30.

[8] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025. Accessed 2026-10-30.

[9] Wei Chen, Yajun Wang, and Yang Yuan. Combinatorial Multi-Armed Bandit: General Framework and Applications. In *International Conference on Machine Learning*, volume 28 of *PMLR*, pages 151–159, 2013. URL https://proceedings.mlr.press/v28/chen13a.html. Accessed 2026-10-30.

[10] Wei Chen, Yajun Wang, Yang Yuan, and Qinshi Wang. Combinatorial Multi-Armed Bandit and Its Extension to Probabilistically Triggered Arms, 2014. URL https://arxiv.org/abs/1407.8339. Accessed 2026-10-30.

[11] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting Language Models to Compress Contexts, 2023. URL https://arxiv.org/abs/2305.14788. Accessed 2026-10-30.

[12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory, 2025. URL https://arxiv.org/abs/2504.19413. Accessed 2026-10-30.

[13] Ashwin Gerard Colaco and Nada Lahjouji. What to Keep, What to Forget: A Rate-Distortion View of Memory Compaction in LLMs and Agents, 2026. URL https://arxiv.org/abs/2607.08032. Accessed 2026-10-30.

[14] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114. Accessed 2026-10-30.

[15] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization, 2024. URL https://arxiv.org/abs/2404.16130. Accessed 2026-10-30.

[16] Liyang Fan, Yingcheng Shi, Yongbin Li, Chenghao Sun, Xin Chen, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. VibeMemBench:

Evaluating Memory Systems for Coding Agents on Real Repository Coding Tasks, 2026. URL https://arxiv.org/abs/2609.23570. Accessed 2026-10-30.

[17] Moran Feldman, Zeev Nutov, and Elad Shoham. Practical Budgeted Submodular Maximization, 2020. URL https://arxiv.org/abs/2007.04937. Accessed 2026-10-30.

[18] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context Autoencoder for Context Compression in a Large Language Model, 2023. URL https://arxiv.org/abs/2307.06945. Accessed 2026-10-30.

[19] Manu Ghulyani, Arunabh Singh, Karan Bharadwaj, Ankit Nath, and Suranjan Goswami. PACMS: Submodular Context Selection as a Pluggable Engine for LLM Agents, 2026. URL https://arxiv.org/abs/2606.20047. Accessed 2026-10-30.

[20] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and Fast Retrieval-Augmented Generation, 2024. URL https://arxiv.org/abs/2410.05779. Accessed 2026-10-30.

[21] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.14802. Accessed 2026-10-30.

[22] Yudai Honda, Keita Fujita, Keiichi Zempo, and Shogo Fukushima. Human-Like Remembering and Forgetting in LLM Agents: An ACT-R-Inspired Memory Architecture. In *International Conference on Human-Agent Interaction*, pages 229–237, 2025. doi: 10.1145/3765766.3765803. URL https://doi.org/10.1145/3765766.3765803. Accessed 2026-10-30.

[23] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2507.05257. Year refers to the ICLR 2026 publication. Accessed 2026-10-30.

[24] Mehmet Iscan. PYTHALAB-MERA: Validation-Grounded Memory, Retrieval, and Acceptance Control for Frozen-LLM Coding Agents, 2026. URL https://arxiv.org/abs/2605.08468. Accessed 2026-10-30.

[25] Mehmet Iscan. Learning When to Remember: Risk-Sensitive Contextual Bandits for Abstention-Aware Memory Retrieval in LLM-Based Coding Agents, 2026. URL https://arxiv.org/abs/2604.27283. Accessed 2026-10-30.

[26] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models, 2023. URL https://arxiv.org/abs/2310.05736. Accessed 2026-10-30.

[27] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression, 2023. URL https://arxiv.org/abs/2310.06839. Accessed 2026-10-30.

[28] Prakhar Khatri. Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories, 2026. URL https://arxiv.org/abs/2607.27250. Accessed 2026-10-30.

[29] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70: 39–45, 1999. doi: 10.1016/S0020-0190(99)00031-9. URL https://doi.org/10.1016/S0020-0190(99)00031-9. Accessed 2026-10-30.

[30] Andreas Krause and Carlos Guestrin. A Note on the Budgeted Maximization of Submodular Functions. Technical Report CMU-CALD-05-103, Carnegie Mellon University, 2005. URL http://reports-archive.adm.cs.cmu.edu/anon/cald/CMU-CALD-05-103.pdf. Accessed 2026-10-30.

[31] Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning, 2012. URL https://arxiv.org/abs/1207.6083. Accessed 2026-10-30.

[32] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated. *Trends in Cognitive Sciences*, 20:512–534, 2016. doi: 10.1016/j.tics.2016.05.004. URL https://doi.org/10.1016/j.tics.2016.05.004. Accessed 2026-10-30.

[33] Branislav Kveton, Zheng Wen, Azin Ashkan, and Csaba Szepesvari. Tight Regret Bounds for Stochastic Combinatorial Semi-Bandits, 2014. URL https://arxiv.org/abs/1410.0949. Accessed 2026-10-30.

[34] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 420–429, 2007. doi: 10.1145/1281192.1281239. URL https://doi.org/10.1145/1281192.1281239. Accessed 2026-10-30.

[35] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. ContextBench: A Benchmark for Context Retrieval in Coding Agents, 2026. URL https://arxiv.org/abs/2602.05892. Accessed 2026-10-30.

[36] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM Knows What You are Looking for Before Generation, 2024. URL https://arxiv.org/abs/2404.14469. Accessed 2026-10-30.

[37] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2402.17753. Accessed 2026-10-30.

[38] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102:419–457, 1995. doi: 10.1037/0033-295X.102.3.419. URL https://doi.org/10.1037/0033-295X.102.3.419. Accessed 2026-10-30.

[39] Michel Minoux. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques*, Lecture Notes in Control and Information Sciences, pages 234–243. 1978. doi: 10.1007/BFb0006528. URL https://doi.org/10.1007/BFb0006528. Accessed 2026-10-30.

[40] Kazuki Nakayashiki. When Stale Constraints Go Unchecked: Budgeted Verification Failures in Inherited Agent Memory, 2026. URL https://arxiv.org/abs/2608.25553. Accessed 2026-10-30.

[41] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971. URL https://doi.org/10.1007/BF01588971. Accessed 2026-10-30.

[42] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory, 2025. URL https://arxiv.org/abs/2509.25140. Accessed 2026-10-30.

[43] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. URL https://arxiv.org/abs/2310.08560. Accessed 2026-10-30.

[44] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression, 2024. URL https://arxiv.org/abs/2403.12968. Accessed 2026-10-30.

[45] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. URL https://arxiv.org/abs/2304.03442. UIST 2023; DOI 10.1145/3586183.3606763. Accessed 2026-10-30.

[46] Chao Peng, Bin Wang, Zhilei Long, and Jinfang Sheng. AdaGReS:Adaptive Greedy Context Selection via Redundancy-Aware Scoring for Token-Budgeted RAG, 2025. URL https://arxiv.org/abs/2512.25052. Accessed 2026-10-30.

[47] Aleksandar Petrov, Mark Sandler, Andrey Zhmoginov, Nolan Miller, and Max Vladymyrov. Long Context In-Context Compression by Getting to the Gist of Gisting, 2025. URL https://arxiv.org/abs/2504.08934. Accessed 2026-10-30.

[48] Nathanaël Carraz Rakotonirina, Mohammed Hamdy, Jon Ander Campos, Lucas Weber, Alberto Testoni, Marzieh Fadaee, Sandro Pezzelle, and Marco Del Tredici. From Tools to Teammates: Evaluating LLMs in Multi-Session Coding Interactions, 2025. URL https://arxiv.org/abs/2502.13791. Accessed 2026-10-30.

[49] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025. URL https://arxiv.org/abs/2501.13956. Accessed 2026-10-30.

[50] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Text REtrieval Conference (TREC-3)*, pages 109–126, 1994. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/okapi_trec3.pdf. Accessed 2026-10-30.

[51] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval, 2024. URL https://arxiv.org/abs/2401.18059. Accessed 2026-10-30.

[52] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL https://arxiv.org/abs/2303.11366. Accessed 2026-10-30.

[53] Baixi Sun, Le Chen, Anjir Ahmed Chowdhury, Xiaolong Ma, Chih-Hsuan Yang, Mingze Xia, Syed Zawad, Sheng Di, Rajkumar Kettimuthu, Huihuo Zheng, Rajeev Thakur, Venkatram Vishwanath, and Feng Yan. LAM: Efficient Lossy Agent Memory Framework With A Retrieval-Score Error Bound, 2026. URL https://arxiv.org/abs/2609.32256. Accessed 2026-10-30.

[54] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts, 2019. URL https://arxiv.org/abs/1909.13231. Accessed 2026-10-30.

[55] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (Learn at Test Time): RNNs with Expressive Hidden States, 2024. URL https://arxiv.org/abs/2407.04620. Accessed 2026-10-30.

[56] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32:41–43, 2004. doi: 10.1016/S0167-6377(03)00062-2. URL https://doi.org/10.1016/S0167-6377(03)00062-2. Accessed 2026-10-30.

[57] Sho Takemori, Masahiro Sato, Takashi Sonoda, Janmajay Singh, and Tomoko Ohkuma. Submodular Bandit Problem Under Multiple Constraints, 2020. URL https://arxiv.org/abs/2006.00661. PMLR 124 (UAI 2020). Accessed 2026-10-30.

[58] Jing Tang, Xueyan Tang, Andrew Lim, Kai Han, Chongshou Li, and Junsong Yuan. Revisiting Modified Greedy Algorithm for Monotone Submodular Maximization with a Knapsack Constraint, 2020. URL https://arxiv.org/abs/2008.05391. Accessed 2026-10-30.

[59] Zhen Tao, Jinxiang Zhao, Peng Liu, Dinghao Xi, Yanfang Chen, Wei Xu, and Zhiyu Li. MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts, 2026. URL https://arxiv.org/abs/2605.20926. Accessed 2026-10-30.

[60] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J Ross Mitchell. Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2510.27246. Accessed 2026-10-30.

[61] Sentence Transformers. sentence-transformers/all-MiniLM-L6-v2 model card, 2026. URL https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Model card. Accessed 2026-10-30.

[62] Stephan Tulkens and Thomas van Dongen. Model2Vec: Fast State-of-the-Art Static Embeddings, 2024. URL https://github.com/MinishLab/model2vec. Repository. Accessed 2026-10-30.

[63] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL https://arxiv.org/abs/2305.16291. Accessed 2026-10-30.

[64] Zheng Wen, Branislav Kveton, and Azin Ashkan. Efficient Learning in Large-Scale Combinatorial Semi-Bandits, 2014. URL https://arxiv.org/abs/1406.7443. Accessed 2026-10-30.

[65] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.10813. Accessed 2026-10-30.

[66] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks, 2023. URL https://arxiv.org/abs/2309.17453. Accessed 2026-10-30.

[67] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents, 2025. URL https://arxiv.org/abs/2502.12110. Accessed 2026-10-30.

[68] Baosheng Yu, Meng Fang, and Dacheng Tao. Linear Submodular Bandits with a Knapsack Constraint. In *AAAI Conference on Artificial Intelligence*, volume 30, 2016. doi: 10.1609/aaai.v30i1.10154. URL https://doi.org/10.1609/aaai.v30i1.10154. Accessed 2026-10-30.

[69] Boqin Yuan, Yue Su, and Kun Yao. Diagnosing Retrieval vs. Utilization Bottlenecks in LLM Agent Memory, 2026. URL https://arxiv.org/abs/2603.02473. Accessed 2026-10-30.

[70] Yisong Yue and Carlos Guestrin. Linear Submodular Bandits and their Application to Diversified Retrieval. In *Advances in Neural Information Processing Systems 24*, pages 2483–2491, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/33ebd5b07dc7e407752fe773eed20635-Abstract.html. Accessed 2026-10-30.

[71] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models, 2025. URL https://arxiv.org/abs/2510.04618. Accessed 2026-10-30.

[72] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory, 2026. URL https://arxiv.org/abs/2601.03192. Accessed 2026-10-30.

[73] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models, 2023. URL https://arxiv.org/abs/2306.14048. Accessed 2026-10-30.

[74] Ziyi Zhang, Shuang Cui, Haotian Zhang, and Xiaoyu Wang. PC-SubMax: Efficient Prompt Compression via Regularized Submodular Maximization, 2026. URL https://arxiv.org/abs/2609.32474. Accessed 2026-10-30.

[75] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A Benchmark for Context Learning in Coding, 2026. URL https://arxiv.org/abs/2602.08316. Accessed 2026-10-30.

[76] Mingxi Zou, Zhihan Guo, Langzhang Liang, Zhuo Wang, Qifan Wang, Qingsong Wen, Irwin King, Lizhen Qu, and Zenglin Xu. Remember the Decision, Not the Description: A Rate-Distortion Framework for Agent Memory, 2026. URL https://arxiv.org/abs/2605.10870. Accessed 2026-10-30.

A Proofs

Proof of Lemma 1. For a facet f, $g_f(S) = 1 - \prod_{m \in S}(1 - u_m)$ with $u_m = w(m)a_{mf} \in [0, 1]$. Adding m to S gains $u_m \prod_{n \in S}(1-u_n)$, which is non-negative and non-increasing in S. So g_f is monotone submodular. The additive term is modular with non-negative coefficients. Each term $S \mapsto \max_{m \in S} s(c, m)$ (with $\max \emptyset = 0$) is the facility-location function, which is monotone submodular. Non-negative combinations preserve both properties, and $F(\emptyset) = 0$.

Coordinate monotonicity follows because $\prod(1 - w(m)a_{mf})$ decreases in each $w(m)$ and the third term does not depend on w. For the Lipschitz bound, $|\prod_i(1 - u_i) - \prod_i(1 - v_i)| \le \sum_i |u_i - v_i|$ for $u, v \in [0, 1]$ by telescoping. So each g_f moves by at most $\sum_{m \in S} |w(m) - w'(m)|$ as $a_{mf} \le 1$, and $\sum_f \omega_f = 1$. The additive term moves by at most η times the same sum, and the factor $(1 - \gamma)$ gives L. □

Proof of Proposition 1. Take additive U. One item a has score $1 + \epsilon$ and cost B. There are $\lfloor B/c_{\min} \rfloor$ items of score 1 and cost $c_{\min}$. The rule fills with a first and then has no budget, so it earns $1 + \epsilon$ against OPT $\ge \lfloor B/c_{\min} \rfloor$. Let $\epsilon \to 0$. □

Proof of Proposition 2. Let there be k facets of equal weight. Facet 1 has k duplicates, each with modular score $1 + \epsilon$. Each other facet has one item with score 1. All costs equal 1, and $B = k$. A modular rule takes the k duplicates and covers one facet; the optimum takes one item per facet and covers all k. □

Proof of Proposition 3. Walking from newest to oldest, keep each item that fits. Since the later items cost more than B in total, one of them fails to fit, and at that moment the remaining budget is below its cost, which is at most $c(m)$. The remaining budget only shrinks afterwards, so m never fits. A relevance rule does not depend on the items' order in time. □

Proof of Proposition 4. Take two versions with equal text similarity to the query, and let the assignment of the two timestamps be uniform. An invariant score cannot depend on the assignment, so the ranking between the two is independent of which one is current. The outdated version ranks first with probability 1/2, or at least 1/2 under ties broken independently of time. □

Proof of Proposition 5. Let each round have two items with features e_1, e_2, costs 1, $B = 1$, $\rho \equiv 1$ and additive F. For a fixed θ, assume without loss of generality that $\theta_1 \ge \theta_2$; the fixed policy injects e_1 on every round. Take $\theta^\star = (0, 1)$, which is valid since $\mu \in [0, 1]$. The policy earns 0 per round against 1, so its α-regret is αT for each $\alpha > 0$. Theorem 2 with $\varepsilon \equiv 0$ gives $\tilde{O}(\sqrt{T})$ on this instance. □

Proof of Theorem 1. Let O be an optimal set and consider the density run ($\varrho = 1$) as a sequence of considered items. Let $g_1, g_2, \dots$ be the items accepted, in order, and write $G_i = \{g_1, \dots, g_i\}$. If no element of O is rejected, then every element of O is either accepted or left with zero marginal gain at termination, so $F(G) \ge F(G \cup O) \ge F(O)$ and we are done. Otherwise let $e^\star$ be the first element of O rejected, and let ℓ be the number of accepted items before it, so $c(G_\ell) + c(e^\star) > B$.

For $i \le \ell$, when g_i was chosen, every element of $O \setminus G_{i-1}$ was still available, since none had been rejected. By monotonicity and submodularity,

$$F(O) \le F(O \cup G_{i-1}) \le F(G_{i-1}) + \sum_{o \in O \setminus G_{i-1}} \Delta(o \mid G_{i-1}) \le F(G_{i-1}) + B\,\frac{\Delta(g_i \mid G_{i-1})}{c(g_i)}.$$

The same inequality holds with $e^\star$ in the role of $g_{\ell+1}$ and $G_{\ell+1} := G_\ell \cup \{e^\star\}$, $c(g_{\ell+1}) := c(e^\star)$, since $e^\star$ had maximum density when chosen. Set $\delta_i = F(O) - F(G_i)$. The inequality reads $\delta_i \le (1 - c(g_i)/B)\,\delta_{i-1}$. If some $\delta_i \le 0$ with $i \le \ell$ then $F(G) \ge F(O)$ and we are done. Otherwise, using $0 \le 1 - x \le e^{-x}$ on $x = c(g_i)/B \in (0, 1]$,

$$\delta_{\ell+1} \le F(O) \prod_{i=1}^{\ell+1} \Big(1 - \frac{c(g_i)}{B}\Big) \le F(O) \exp\Big(-\frac{\sum_{i \le \ell+1} c(g_i)}{B}\Big) \le \frac{F(O)}{e},$$

because $\sum_{i \le \ell+1} c(g_i) = c(G_\ell) + c(e^\star) > B$. So $F(G_\ell \cup \{e^\star\}) \ge (1 - 1/e)F(O)$. By subadditivity $F(G_\ell) + F(\{e^\star\}) \ge (1 - 1/e)F(O)$, so one of the two terms is at least $\frac{1}{2}(1 - 1/e)F(O)$. Now $F(G) \ge F(G_\ell)$ by monotonicity, and $F(\{e^\star\})$ is at most the value of the best singleton, which fits since $c(e^\star) \le B$. The algorithm returns the maximum over a set of runs that includes both. □

Proof of Proposition 6. For any feasible O, monotonicity and submodularity give $F(O) \le F(S) + \sum_{m \in O \setminus S} \Delta(m \mid S)$. The indicator of $O \setminus S$ is feasible for the linear program, whose optimum is reached by the fractional-knapsack rule. □

Lemma 2 (Batched elliptical potential)**.** *Let $\lambda \ge K$, $V_1 = \lambda I$, $V_{t+1} = V_t + \sum_{i \in A_t} x_i x_i^\top$ with $|A_t| \le K$ and $\|x_i\| \le 1$. Then $\sum_{t \le T} \sum_{i \in A_t} \|x_i\|^2_{V_t^{-1}} \le 2d \log(1 + KT/(d\lambda))$.*

Proof. Let $B_t = V_t^{-1/2} (\sum_{i \in A_t} x_i x_i^\top) V_t^{-1/2}$, positive semidefinite with $\operatorname{tr} B_t = \sum_{i \in A_t} \|x_i\|^2_{V_t^{-1}} \le K/\lambda \le 1$. So its eigenvalues b_j lie in $[0,1]$, and $\log\det(I + B_t) = \sum_j \log(1 + b_j) \ge \sum_j b_j \log 2 \ge \frac{1}{2} \operatorname{tr} B_t$. Since $\det V_{t+1} = \det V_t \det(I + B_t)$, summing gives $\sum_t \operatorname{tr} B_t \le 2 \log(\det V_{T+1} / \det V_1)$. Finally, $\operatorname{tr} V_{T+1} \le d\lambda + KT$ and AM–GM give $\det V_{T+1} \le (\lambda + KT/d)^d$. □

Proof of Theorem 2. Good event. Order items within a round arbitrarily, and take the filtration that reveals a round's Z values before its rewards. Then $X_{t,m} = Z_{t,m} x_{t,m}$ is predictable, $Z_{t,m} \xi_{t,m}$ is conditionally R-sub-Gaussian, and items with $Z = 0$ contribute nothing. The self-normalized bound [1, Theorem 2], applied to $\theta^\star - \theta_0$ with targets $y - x^\top \theta_0$, gives with probability at least $1 - \delta/4$ for all t: $\|\hat\theta_t - \theta^\star\|_{V_t} \le \beta_T$. The sequence has at most KT terms. So $|x^\top \hat\theta_t - x^\top \theta^\star| \le \beta_T \|x\|_{V_t^{-1}}$ for every x by Cauchy–Schwarz. Combined with the validity event (probability at least $1 - \delta/2$), and writing $\bar\rho_{t,m} = \min(1, \hat\rho_{t,m} + \varepsilon_{t,m})$ and $\bar\mu_{t,m}$ for the clipped upper confidence bound,

$$\rho_{t,m} \le \bar\rho_{t,m} \le \rho_{t,m} + 2\varepsilon_{t,m}, \qquad \mu_{t,m} \le \bar\mu_{t,m} \le \mu_{t,m} + b_{t,m}, \quad b_{t,m} := \min(1, 2\beta_T \|x_{t,m}\|_{V_t^{-1}}).$$

Clipping preserves the lower bound as $\mu_{t,m} \in [0,1]$. In particular $w_t^\star \le \bar w_t$ coordinate-wise.

Per-round regret. Let $S_t^\star$ be optimal for $w_t^\star$. By coordinate monotonicity, the oracle property, and the Lipschitz property,

$$\alpha F_t(S_t^\star; w_t^\star) \le \alpha F_t(S_t^\star; \bar w_t) \le F_t(S_t; \bar w_t) \le F_t(S_t; w_t^\star) + L \sum_{m \in S_t} (\bar w_{t,m} - w_{t,m}^\star).$$

Moreover, $\bar w - w^\star = \bar\rho\bar\mu - \rho\mu = \rho(\bar\mu - \mu) + (\bar\rho - \rho)\bar\mu \le \rho\, b + 2\varepsilon$.

Summation. $b_{t,m}$ is determined by the history before round t, and $\mathbb{E}[Z_{t,m} \mid \text{history}] = \rho_{t,m}$, while Z values within a round may be dependent. So $D_t = \sum_{m \in S_t} (\rho_{t,m} - Z_{t,m}) b_{t,m}$ is a martingale difference sequence with $|D_t| \le K$, and Azuma–Hoeffding gives, with probability at least $1 - \delta/4$, $\sum_t \sum_{m \in S_t} \rho_{t,m} b_{t,m} \le \sum_t \sum_{m \in S_t} Z_{t,m} b_{t,m} + K\sqrt{2T \log(4/\delta)}$. The items with $Z = 1$ are exactly the ones entering V. By Cauchy–Schwarz and Lemma 2,

$$\sum_t \sum_{m \in S_t} Z_{t,m} b_{t,m} \le 2\beta_T \sqrt{KT \sum_t \sum_{m : Z = 1} \|x_{t,m}\|^2_{V_t^{-1}}} \le 2\beta_T \sqrt{2dKT \log(1 + KT/(d\lambda))}.$$

A union bound over the three events finishes the proof, and $\sum_t \sum_{m \in S_t} 2\varepsilon_{t,m}$ is the remaining term. □

Proof of Proposition 7. Write $\lambda = \lambda_k$. The change count $N(s)$ has intensity $\lambda Y(s)$, where Y is the predictable number of items at risk, and $E(s) = \int_0^s Y$. For any fixed real θ, $M_\theta(s) = \exp(\theta N(s) - \lambda(e^\theta - 1) E(s))$ is a non-negative local martingale with $M_\theta(0) = 1$, hence a supermartingale, so $\Pr[M_\theta \ge 2/\delta] \le \delta/2$ at the stopping time by Markov's inequality and optional stopping.

Let $\ell = \log(2/\delta)$ and $\theta = \sqrt{\ell/(\lambda E_0)} = \eta/2 \le \frac{1}{4}$. Taking $+\theta$ and using $(e^\theta - 1)/\theta \le 1 + \theta$ on $[0,1]$: $N \le \lambda E(1 + \theta) + \ell/\theta$. Taking $-\theta$ and using $(1 - e^{-\theta})/\theta \ge 1 - \theta$: $N \ge \lambda E(1 - \theta) - \ell/\theta$. Dividing by λE and using $E \ge E_0$, with $\ell/(\theta\lambda E_0) = \theta$, gives $|\hat\lambda/\lambda - 1| \le 2\theta = \eta$.

Put $s = \hat\lambda/\lambda \in [\frac{1}{2}, \frac{3}{2}]$. By the mean value theorem, for some σ between 1 and s, so $\sigma \ge \frac{1}{2}$, we have $|e^{-s\lambda a} - e^{-\lambda a}| = |s - 1|\, \lambda a\, e^{-\sigma\lambda a} \le \eta/(e\sigma) \le 2\eta/e$, using $\sup_{u \ge 0} u e^{-\sigma u} = 1/(e\sigma)$, for all a at once. □

Proof of Proposition 8. Let each round have m items with equal features, $\mu = 1$, $c = 1$, $B = 1$ and $F(S; w) = \sum_{i \in S} w_i$, with the index of the valid item uniform and independent across rounds. The information a blind learner gets from past rounds is independent of the current valid index, so whichever item it picks is valid with probability $1/m$ and $\mathbb{E} F_t(S_t) = 1/m$, while $\mathrm{OPT}_t = 1$. □

Proof of Proposition 9. $\int_0^\infty e^{-\lambda a} \frac{\beta^\alpha \lambda^{\alpha-1} e^{-\beta\lambda}}{\Gamma(\alpha)}\, d\lambda = (\beta/(\beta + a))^\alpha$. The logarithm is $-\alpha \log(1 + a/\beta) = -\alpha \log a + \alpha \log \beta + o(1)$. A single access at age a has ACT-R activation $B = \log a^{-d} = -d \log a$. □

Proof of Theorem 3. Φ is monotone submodular (facility location), $\Phi(\emptyset) = 0$ and $\Phi(X) = 1$. For the greedy sequence with density-maximal g_i, as in the proof of Theorem 1, $\Phi(Z^\star) - \Phi(Z_{i-1}) \le R^\star \Delta_i / c(g_i)$, so $d_i := \Phi(Z^\star) - \Phi(Z_i) \le (1 - c(g_i)/R^\star) d_{i-1} \le e^{-c(g_i)/R^\star} d_{i-1}$ whenever $d_{i-1} > 0$, using $1 - x \le e^{-x}$ for all real x. If $d_{i-1} \le 0$ for some i, then $\Phi(Z_\ell) \ge \Phi(Z^\star)$ by monotonicity. In every case $\Phi(Z_\ell) \ge (1 - e^{-R_\ell / R^\star}) \Phi(Z^\star)$. Substituting $\Phi = 1 - D$ gives the first claim.

For the second, unit vectors satisfy $\|e_x - e_z\|^2 = 2(1 - \langle e_x, e_z \rangle)$. So $|\langle e_q, e_x - e_z \rangle| \le \sqrt{2(1 - s(x, z))}$, which bounds the score change by $\ell_{\mathrm{u}} \sqrt{2(1 - s(x, z(x)))}$. Average over p and apply Jensen's inequality to the square root, noting that $\mathbb{E}_p[1 - s(x, z(x))] = D(Z_\ell)$. □

Proof of Theorem 4. Condition on r and let T be the time until the first gap of length τ in a Poisson process of rate r started at the arrival. Let $A \sim \mathrm{Exp}(r)$ be the first access. If $A > \tau$ then $T = \tau$, otherwise $T = A + T'$ with T' an independent copy of T by the memoryless property. So $\mathbb{E}T = \mathbb{E}\min(A, \tau) + (1 - e^{-r\tau}) \mathbb{E}T$. With $\mathbb{E}\min(A, \tau) = (1 - e^{-r\tau})/r$, this gives $\mathbb{E}T = (e^{r\tau} - 1)/r$.

The pairs (arrival time, lifetime) form a marked Poisson process with i.i.d. marks. By the marking theorem, the number of items alive at t is Poisson with mean $a \int_0^t \Pr[L > u]\, du$, which converges to $a\, \mathbb{E}L$ under the finiteness assumption. □